Virtual Environment (Lecture No. 40)

Part 1 — Set up the virtual environment

1

Create a project folder on drive D:

We want a dedicated folder so all our project files and packages stay in one place — separate from the rest of the system.

What to doManually create a folder named **python-venv** inside **D:** using File Explorer.

2

Open Command Prompt and navigate to that folder

We need to be inside the folder before running any Python commands. D: switches drive, cd changes directory.

D:cd python-venvdir

Running dir confirms you are in the right folder.

3

Create the virtual environment

What is a virtual environment? It is an isolated Python workspace. Packages you install here don't affect the rest of your computer, and you can have different versions of the same package in different environments.

python -m venv myenv

This creates a folder called myenv inside python-venv.

4

Activate the virtual environment

Why activate? Just creating the environment is not enough — you must activate it so that Python and pip point to the environment's isolated copies instead of the system-wide ones.

myenv\Scripts\activate(myenv) D:\python-venv>

The (myenv) prefix in the prompt means the environment is active. Everything you install now goes only into myenv.

Part 2 — Install packages and verify isolation

5

Confirm pandas is not yet installed in the environment

Before installing, we verify that the environment is truly isolated and does not inherit packages from the system Python.

(myenv)pythonimport pandas as pdModuleNotFoundError: No module named 'pandas'

This error is expected and good — it proves the environment is isolated.

exit()

6

Install a specific version of pandas

Why a specific version? Different projects may need different library versions. Virtual environments let you pin the exact version each project requires, so nothing breaks when you upgrade for another project.

(myenv)pip3 install pandas==2.3.3

To install the *latest* version instead, just use pip3 install pandas (no version number).

7

Confirm the correct version is installed

(myenv)pythonimport pandas as pdpd.__version__'2.3.3'exit()

Part 3 — Deactivate and see the difference

8

Deactivate and compare versions

Why deactivate? To prove the environment truly is isolated. After deactivating, Python falls back to the system-wide installation with a completely different package set.

(myenv)deactivatepythonimport pandas as pdpd.__version__'3.0.2' ← system version, not our 2.3.3exit()

Different version shown = environment is perfectly isolated.

9

Re-activate any time you want to work on the project

myenv\Scripts\activate(myenv) D:\python-venv>

Activate = step into the isolated environment. Deactivate = step back out. You can do this as many times as needed.

Part 4 — Create a project file

10

Create a Python file inside the project folder

touch is a Linux command and does not work on Windows. Use type nul instead — it creates an empty file.

(myenv)type nul > main.py

11

Run the file — notice the version changes with activation

Without env activepython main.py3.0.2

With env active(myenv) python main.py2.3.3

Part 5 — requirements.txt (share your environment with others)

12

Install any additional packages your project needs

(myenv)pip install dateutils**(myenv)**pip install lxml**(myenv)**pip install openpyxl

13

Export all installed packages to requirements.txt

Why? When you share your project or set it up on a new machine, this file tells Python exactly which packages (and which versions) are needed — like a recipe list for your environment.

(myenv)pip freeze > requirements.txt

A file called requirements.txt is created in your folder listing every package and its version.

To just view the list on screen (without saving), run: pip freeze

14

Install all packages from requirements.txt on a new machine

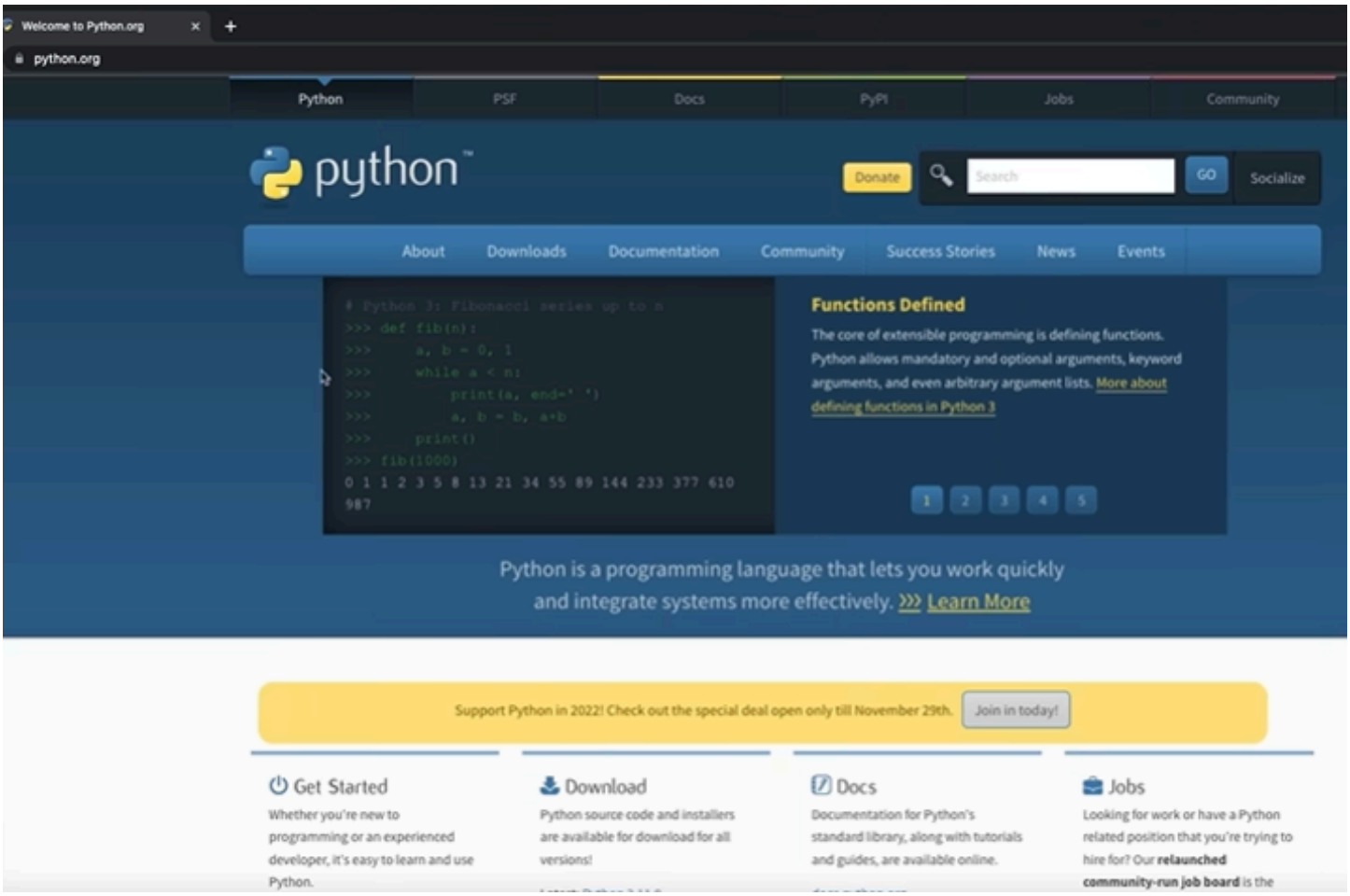
When to use this: When a teammate clones your project, or you're setting up a server, run this one command to install every package automatically instead of installing them one by one.

(myenv)pip install -r requirements.txt

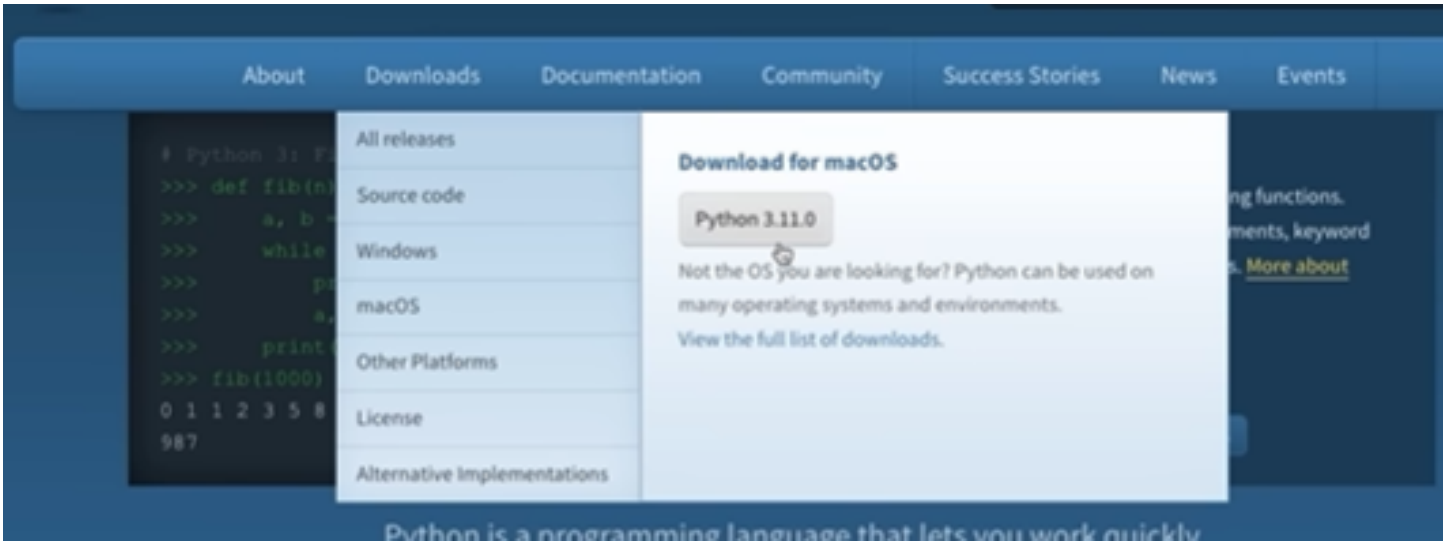
Lecture No. 1 :

How to Download Python ?

→ Search Python on Google Crome.

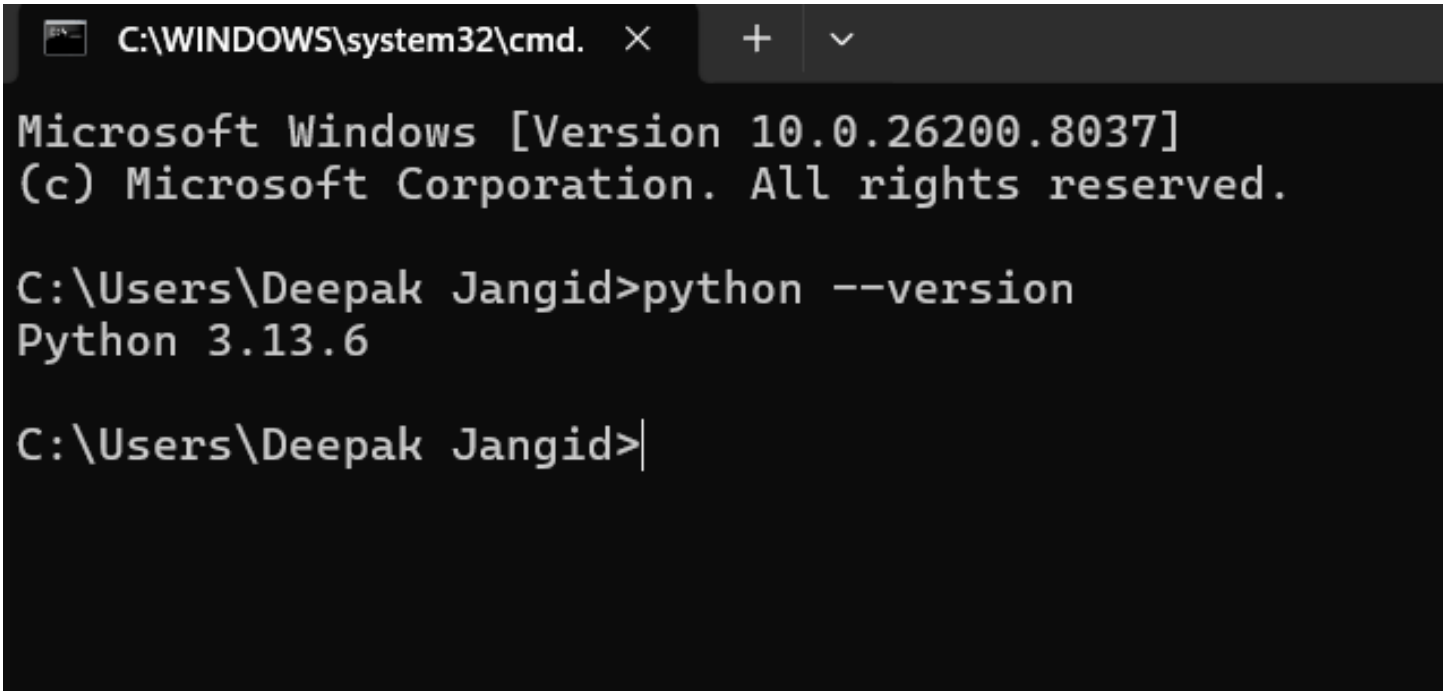


Tap on Download and download it (it is automatically recommended as per the device)



and Download it.

Check in cmd with “python --version”

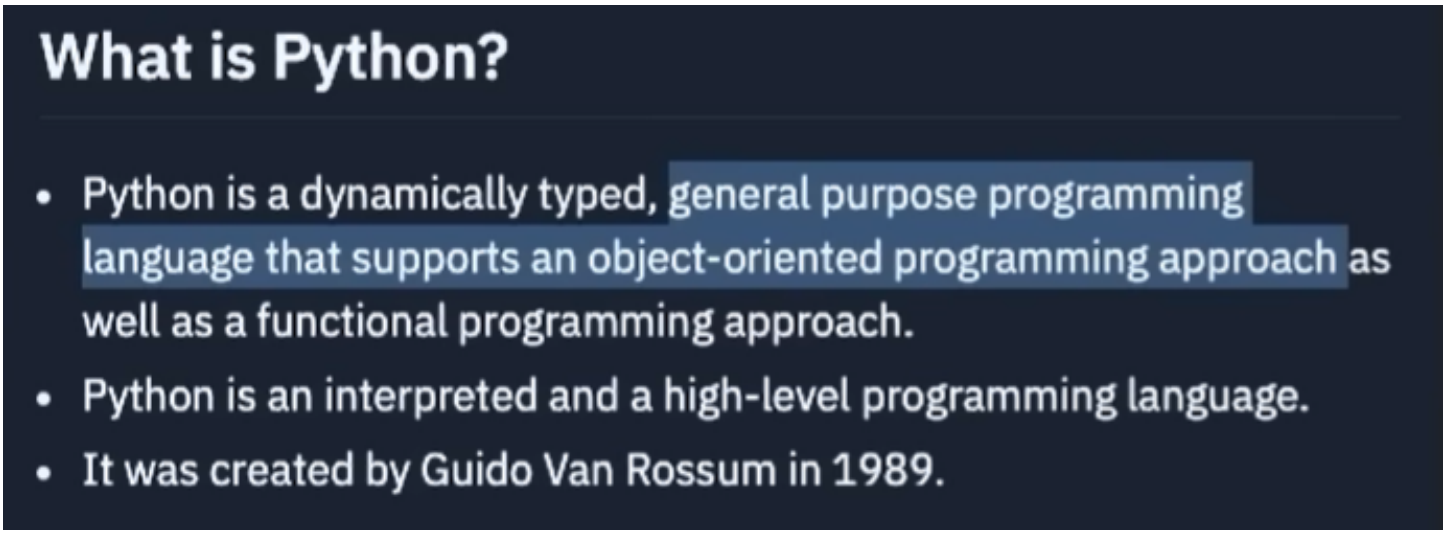


Programming Language :

Programming is a way for us to tell computers what to do.

Programming is the process of creating a set of instructions that tell a computer how to perform a task. It involves analyzing a problem, designing step-by-step algorithms, and writing them in a language the computer understands to build software, websites, and applications. [1, 2, 3, 4, 5].

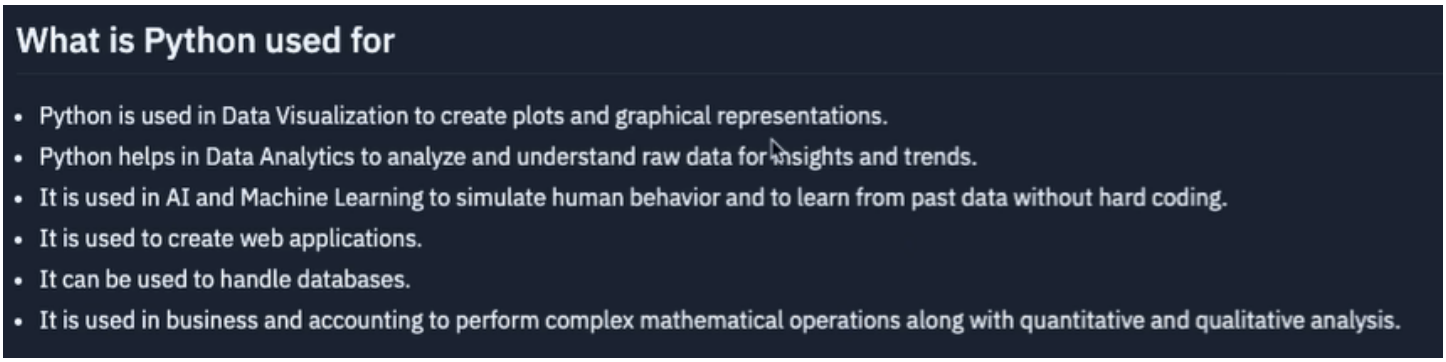
Python :



A dynamically typed language is a programming language where type checking is performed at **runtime** rather than at compile-time.

An **interpreted language** is a programming language where the computer translates and executes the source code line-by-line in real-time, using a program called an **interpreter**.

A high-level language is a programmer-friendly programming language designed to be easily read, written, and understood by humans. By using natural English-like syntax and abstraction.



Lecture No. 2 :

=> Modules

=> Pip

Modules :

→ Basically used to borrow other's code.

→ We can use code which is written by another coder in our python code using Modules.

→ In Python, a **module** is a file that contains ready-made functions, classes, and code which we can use in our program.

Instead of writing everything from scratch, we can import modules and use their features directly.

→ Two types of Modules :

How modules help us

- Save time
- Reduce code length
- Make programming easier
- Reuse code again and again
- Provide powerful ready-made features

→ By which we can easily work on heavy projects and build them.

i) Build in Modules (install using pip)

ii) External Modules (install using pip)

External Modules : are modules made by other developers that are not built into python by default. We must install them before using them.

Example → pyttsx3 (this module converts text into speech)

install :

pip install pyttsx3

Program :

import pyttsx3

engine = pyttsx3.init()

engine.say(“Hello Deepak”)

engine.runAndWait()

Output :

Computer speaks : “Hello Deepak”

Other External Modules Examples :

numpy → Mathematics

pandas → Data analysis

pygame → Games

requests → API & web requests

flask → Web Development

Build in Module : in python are modules that already come with Python. We do not need to install them.

Example → math (used for mathematical operations)

import math

print(math.sqrt(25))

Output : 5.0

Other common build-in modules :

random → Generate random numbers

os → Work with files/folders

datetime → Date & Time

time → Delay and timing

statistics → Mean, Median and Mode

Another Example :

import random

print(random.randint(1,10))

this prints a random number between 1 and 10

---

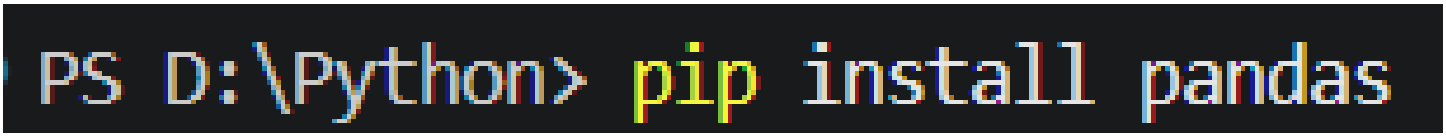
pip is the **standard package manager for Python**. It is used to install, upgrade, and manage additional libraries and dependencies that are not included in the standard Python distribution. [1, 2, 3, 4]

Practical :

Install Pandas.

Commands to use in terminal in VS Code :

pip install pandas



Install skilearn

pip install scikit-learn

Build in module, no need to install directly use it

import hashlib

Install tensorflow

pip install tensorflow
